

AMATH 483 / 583 (Roche) - Homework Set 4

Due Friday May 2, 5pm PT

April 25, 2025

Homework 4 (90 points)

1. (+10) Given matrix $A = \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix}$, evaluate the action of A on the unit balls of $\mathbb{R}^2$ defined by the 1-norm, 2-norm, and ∞ -norm (induced matrix norms). Submit your work and drawings.

①

$$1\text{-norm: } \|A\|_1 = \max_{1 \leq j \leq 2} \sum_{i=1}^2 |a_{ij}|$$

$$\text{column 1 sum: } |1| + |0| = 1$$

$$\text{column 2 sum: } |2| + |2| = 4$$

$$\|A\|_1 = 4$$

unit ball in the 1-norm $\nearrow$

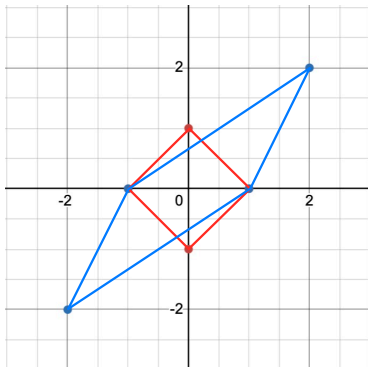
$$B_1 = \{x \in \mathbb{R}^2 : \|x\|_1 \leq 1\}$$

$$= \{(x_1, x_2) : |x_1| + |x_2| \leq 1\}$$

$$\text{extreme points (vertices): } \{(\pm 1, 0), (0, \pm 1)\}$$

$$1\text{-norm unit ball in } \mathbb{R}^2 \text{ is the convex hull of } \underline{\{(1, 0), (-1, 0), (0, 1), (0, -1)\}}$$

$$\begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \underline{\begin{pmatrix} 1 \\ 0 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ 0 \end{pmatrix} = \underline{\begin{pmatrix} -1 \\ 0 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \underline{\begin{pmatrix} 2 \\ 2 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \underline{\begin{pmatrix} 2 \\ -2 \end{pmatrix}}$$



• 1-norm unit ball

• $A \cdot (1\text{-norm unit ball})$

②

$$2\text{-norm: } \|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sqrt{\lambda_{\max}(A^T A)}$$

$$A^T A = \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 2 & 8 \end{pmatrix}$$

$$\begin{aligned} \det(A^T A - \lambda I) &= (1-\lambda)(8-\lambda) - 4 \\ &= 8 - 9\lambda + \lambda^2 - 4 \\ &= \lambda^2 - 9\lambda + 4 \end{aligned}$$

$$\lambda^2 - 9\lambda + 4 = 0$$

$$\begin{aligned} \lambda_1 &= \frac{9 \pm \sqrt{81-16}}{2} & \sigma_1 &= 2.9208 \\ \lambda_2 &= \frac{9 \pm \sqrt{81-16}}{2} & \sigma_2 &= 0.6847 \end{aligned}$$

$$\lambda_{\max} \approx 8.5311$$

$$\sqrt{\lambda_{\max}(A^T A)} = \sqrt{\frac{9 + \sqrt{65}}{2}}$$

$$\|A\|_2 = \sqrt{\frac{9 + \sqrt{65}}{2}} \approx 2.9208$$

$$\lambda_2 = \frac{9 - \sqrt{65}}{2} \approx 0.47$$

$$A^T A - \lambda_2 I = \begin{pmatrix} 1 - 0.47 & 2 \\ 2 & 8 - 0.47 \end{pmatrix}$$

$$(A^T A - \lambda_2 I) \vec{v}_2 = 0$$

$$\begin{aligned} 0.53 v_1 + 2v_2 &= 0 \\ 2v_2 &= -0.53 v_1 \\ v_2 &= -0.265 v_1 \end{aligned}$$

normalize to unit length

$$\vec{v}_2 = \frac{\vec{v}_2}{\|\vec{v}_2\|} \approx \begin{pmatrix} 0.966 \\ -0.257 \end{pmatrix}$$

$$V = \begin{pmatrix} 0.257 & 0.966 \\ 0.966 & -0.257 \end{pmatrix} \quad \Sigma = \begin{pmatrix} 2.9208 & 0 \\ 0 & 0.6847 \end{pmatrix}$$

$$A = U \Sigma V^T$$

$$U = A V \Sigma^{-1}$$

$$U = \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0.257 & 0.966 \\ 0.966 & -0.257 \end{pmatrix} \begin{pmatrix} 1/2.9208 & 0 \\ 0 & 1/0.6847 \end{pmatrix}$$

$$\begin{aligned} U &= \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0.257 & 0.966 \\ 0.966 & -0.257 \end{pmatrix} \begin{pmatrix} 0.3424 & 0 \\ 0 & 1.4605 \end{pmatrix} \\ &= \begin{pmatrix} 2.189 & 0.452 \\ 1.932 & -0.514 \end{pmatrix} \begin{pmatrix} 0.3424 & 0 \\ 0 & 1.4605 \end{pmatrix} \end{aligned}$$

$$= \begin{pmatrix} 0.75 & 0.66 \\ 0.662 & -0.751 \end{pmatrix}$$

$$A^T A - \lambda_1 I = \begin{pmatrix} 1 - 8.5311 & 2 \\ 2 & 8 - 8.5311 \end{pmatrix}$$

$$(A^T A - \lambda_1 I) \vec{v}_1 = 0$$

$$(A^T A - \lambda_1 I) \vec{v}_1 = \begin{pmatrix} -7.5311 & 2 \\ 2 & -0.5311 \end{pmatrix}$$

$$-7.5311 v_1 + 2 v_2 = 0$$

$$2 v_2 = 7.5311 v_1$$

$$v_2 = 3.7656 v_1$$

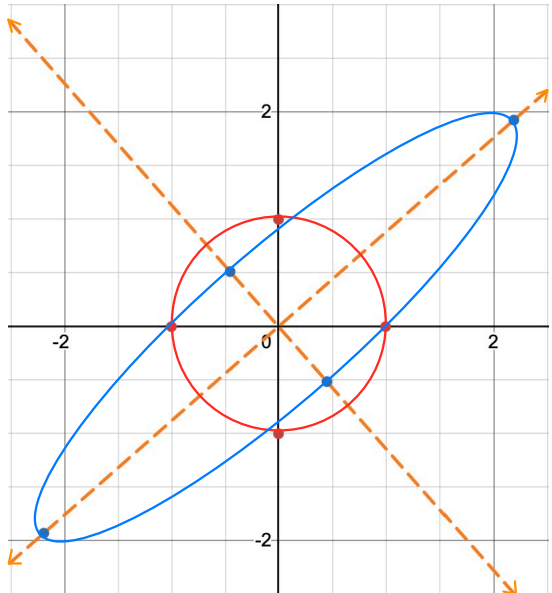
$$\vec{v}_1 = \begin{pmatrix} 1 \\ 3.7656 \end{pmatrix}$$

normalize to unit length

$$\vec{v}_1 = \frac{\vec{v}_1}{\|\vec{v}_1\|} = \frac{1}{\sqrt{1^2 + 3.7656^2}} \begin{pmatrix} 1 \\ 3.7656 \end{pmatrix} \approx \begin{pmatrix} 0.257 \\ 0.966 \end{pmatrix}$$

$$\vec{v}_1 \approx \begin{pmatrix} 0.257 \\ 0.966 \end{pmatrix}$$

$$B_2 = \{x : \sqrt{x_1^2 + x_2^2} \leq 1\}$$



• 2-norm unit ball

• A · (2-norm unit ball)

③ ∞ - norm (induced matrix norms)

$$\|A\|_{\infty} = \max_{1 \leq i \leq 2} \sum_{j=1}^2 |a_{ij}|$$

$$\text{Row 1 sum: } |1| + |2| = 3$$

$$\text{Row 2 sum: } |0| + |2| = 2$$

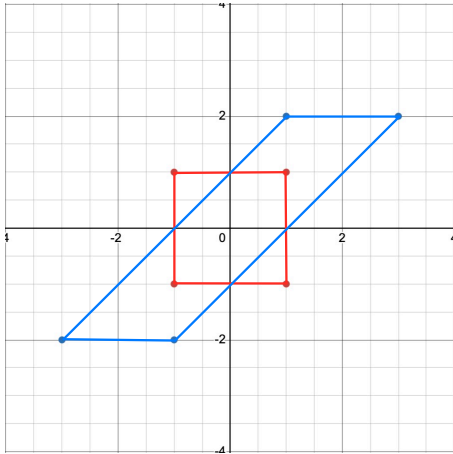
$$\|A\|_{\infty} = 3$$

$$\begin{aligned} B_{\infty} &= \{x : \|x\|_{\infty} \leq 1\} = \{x : \max(|x_1|, |x_2|) \leq 1\} \\ &= \{x : |x_1| \leq 1, |x_2| \leq 1\} \end{aligned}$$

extreme points (vertices): $\{(\pm 1, \pm 1)\}$

convex hull of $\{(1, 1), (-1, 1), (1, -1), (-1, -1)\}$

$$\begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \underline{\begin{pmatrix} 3 \\ 2 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \underline{\begin{pmatrix} 1 \\ 2 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \underline{\begin{pmatrix} -1 \\ 2 \end{pmatrix}} \quad \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \end{pmatrix} = \underline{\begin{pmatrix} -3 \\ 2 \end{pmatrix}}$$



• ∞ - norm unit ball

• $A \cdot (\infty \text{ - norm unit ball})$

2. (+20) Compiler Optimization of Matrix Multiplication Loop Permutations. Implement C++ templated `gemm`, $C \leftarrow \alpha AB + \beta C$ ($A \in \mathbb{T}^{m \times p}$, $B \in \mathbb{T}^{p \times n}$, $C \in \mathbb{T}^{m \times n}$, $\alpha, \beta \in \mathbb{T}$) for $\{kij\}$ and $\{jki\}$ loop permutations using the specifications provided here:

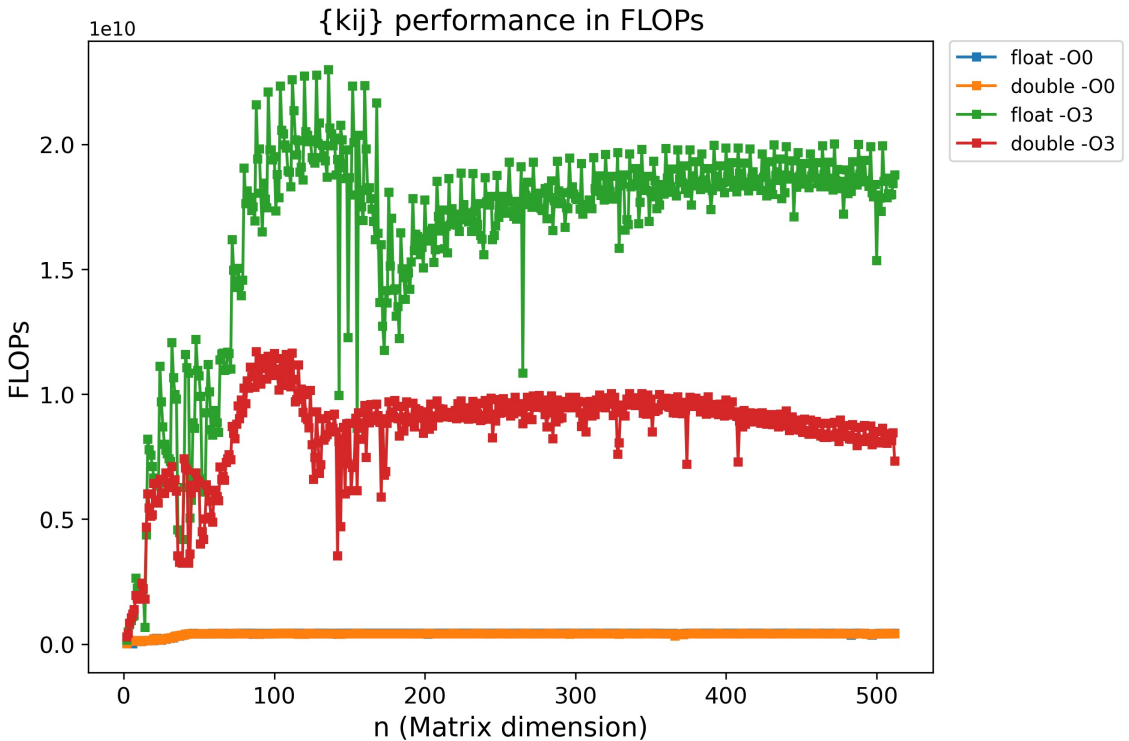
```

• template<typename T>
  void mm_jki(T a, const std::vector<T>& A, const std::vector<T>& B, T b,
    std::vector<T>& C, int m, int p, int n);

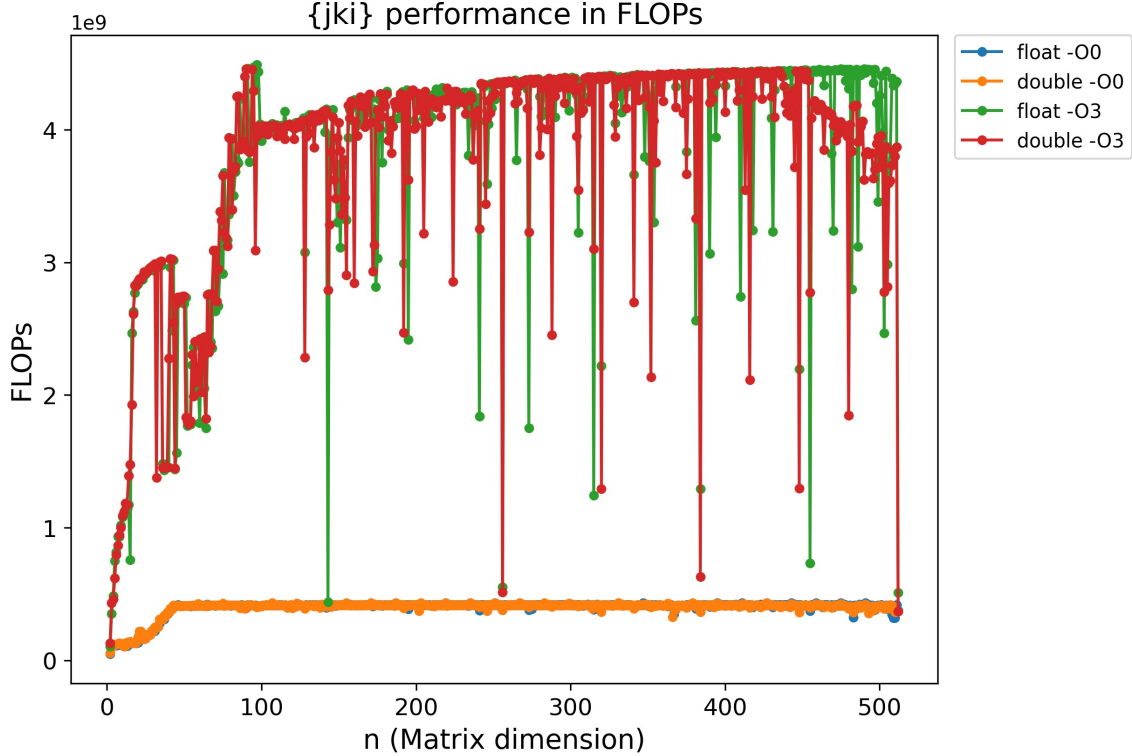
• template<typename T>
  void mm_kij(T a, const std::vector<T>& A, const std::vector<T>& B, T b,
    std::vector<T>& C, int m, int p, int n);

```

You will explore matrix multiply performance applying compiler optimization levels `-O0` and `-O3` for square matrices of dimension $n = 2$ to $n = 512$, stride one to these functions. For reference, recall the loop order for $C_{i,j} \leftarrow \sum_{k=0}^{p-1} A_{i,k} * B_{k,j} \forall [i = 0, \dots, m-1][j = 0, \dots, n-1]$ is $\{ijk\}$, i outer loop, j middle loop, k inner loop. Let each n be measured $ntrial$ times and plot the average performance in FLOPs (flop count / time(seconds)) for each case versus n , $ntrial \geq 3$. Submit plots for both permutation variants that include both FP32 (*float*) and FP64 (*double*) compiler optimization results.



{jki} performance in FLOPs



4. (+10) **Extremum.** Consider the surface defined by $xy + 2xz = 5\sqrt{5}$ and $(x, y, z) \in \mathbb{R}^3$. Find (a) the coordinate instance(s) affiliated with the minimum distance from a point on the surface to the origin, and (b) the value of the minimum distance. You will find (may safely assume) the domain $[-3, 3] \times [-3, 3] \times [-3, 3] \in \mathbb{R}^3$ holds the correct coordinate instance(s).

norm to minimize :

$$f(x, y, z) = x^2 + y^2 + z^2$$

direction of change (partial derivatives):

$$(2x, 2y, 2z)$$

constraint surface :

$$xy + 2xz = 5\sqrt{5}$$

direction of change (partial derivatives):

$$(y + 2z, x, 2x)$$

The rate of change of f must be a scalar multiple of the rate of change of the constraint.

$$(2x, 2y, 2z) = \lambda(y + 2z, x, 2x)$$

$$\begin{cases} 2x = \lambda(y + 2z) \\ 2y = \lambda x \\ 2z = 2\lambda x \end{cases}$$

$$\lambda = \frac{2y}{x}$$

$$z = \lambda x = 2y \quad x^2 = 5y^2$$

$$2x = \lambda(y + 2y) \quad x = \pm\sqrt{5}y$$

$$2x = \frac{10y^2}{x}$$

$$\begin{cases} x = \pm\sqrt{5}y \\ y \\ z = 2y \end{cases} \quad \begin{cases} \pm\sqrt{5} \\ \pm 1 \\ \pm 2 \end{cases}$$

critical points:

$$(\sqrt{5}, 1, 2)$$

$$(-\sqrt{5}, -1, -2)$$

a) coordinate instance affiliated with the minimum distance from a point on the surface to the origin :

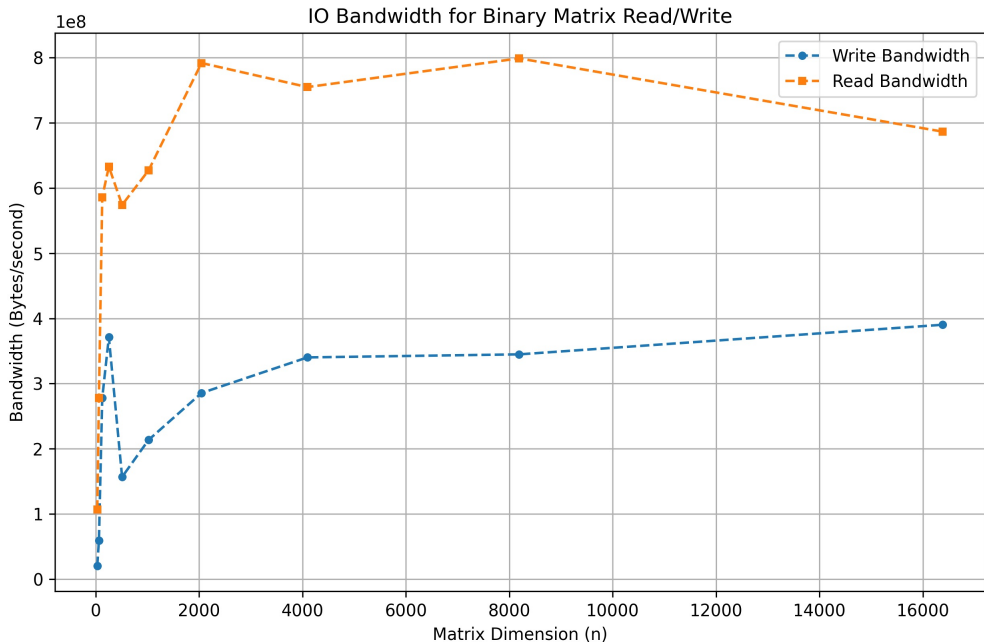
$$(\sqrt{5}, 1, 2), (-\sqrt{5}, -1, -2)$$

b) value of minimum distance :

$$\begin{aligned} \sqrt{(\sqrt{5})^2 + 1^2 + 2^2} &= \sqrt{5+1+4} \\ &= \sqrt{10} \end{aligned}$$

$$\sqrt{10}$$

5. (+10) **IO bandwidth.** Write a C++ function that **writes** type **double** square matrices in column major order to file in binary. Measure the time required to complete the **write** for matrices of dimension 32, 64, 128, ... 16384 (2GB). Write a C++ function that **reads** binary matrices from file to type **double** matrices in memory. Measure the time required to complete each **read** for the same dimensions. (a) Make a single plot of the *read* and *write* measurements with the bandwidth (bytes per second) on the y-axis, and the problem dimension on the x-axis. Submit your plot.



6. (+20) **File access time.** (a) Write C++ functions for the given function declarations that perform *row* and *column* swap operations on a type `double` matrix stored in a file in column major index order. Test the swapping capabilities for correctness. Put the functions you write in file `file_swaps.hpp`. (b) Conduct a performance test for square matrix dimensions 16, 32, 64, 128, ... 8192, measuring the time required to conduct file-based *row* and *column* swaps separately. Let each operation be measured *ntrial* times, $ntrial \geq 3$. Make a single plot of the *row* and *column* swap average times on the y-axis ($\log_{10}(time)$) and the problem dimension on the x-axis. Submit your header file `file_swaps.hpp` and plot. You may find the code snippet helpful.

